

26. Fruit and Color Using Backtracking in Prolog

Aim

To write a Prolog program to identify the **color of a fruit using backtracking**.

Algorithm

1. Define fruits and their colors as facts.
2. Query a color or fruit.
3. Prolog searches for matching facts.
4. On pressing ;, Prolog backtracks and finds the next matching answer.

Program

```
fruit_color(apple, red).
```

```
fruit_color(banana, yellow).
```

```
fruit_color(grapes, green).
```

```
fruit_color(orange, orange).
```

```
fruit_color(strawberry, red).
```

Sample Queries

Query 1: Find the color of Apple

```
?- fruit_color(apple, X).
```

Output

```
X = red.
```

Query 2: Find all red fruits using Backtracking

```
?- fruit_color(X, red).
```

Output

```
X = apple ;
```

```
X = strawberry.
```

Here, when ; is pressed, Prolog **backtracks** and displays the next solution.

Result

Thus, the Prolog program to identify **fruits and their colors using backtracking** was successfully implemented.

27. Breadth First Search (BFS) in Prolog

Aim

To write a Prolog program to implement the **Breadth First Search (BFS)** algorithm.

Algorithm

1. Start with the initial node.
2. Add the node to the queue.
3. Remove the first node from the queue.
4. If it is the goal node, stop.
5. Otherwise, add its unvisited adjacent nodes to the queue.
6. Repeat until the goal is found.

Program

```
% Graph edges
```

```
edge(a, b).
```

```
edge(a, c).
```

```
edge(b, d).
```

```
edge(b, e).
```

```
edge(c, f).
```

```
edge(e, g).
```

```
% BFS
```

```
bfs(Start, Goal, Path) :-
```

```
    bfs_queue([[Start]], Goal, Path).
```

```
bfs_queue([[Goal|Path]|_], Goal, Result) :-
```

```
    reverse([Goal|Path], Result).
```

```
bfs_queue([Path|Paths], Goal, Result) :-
```

```
    Path = [Node|_],
```

```

findall([Next | Path],
        (edge(Node, Next),
         \+ member(Next, Path)),
        NewPaths),
append(Paths, NewPaths, Queue),
bfs_queue(Queue, Goal, Result).

```

Sample Input

?- bfs(a, g, Path).

Sample Output

Path = [a, b, e, g].

Result

Thus, the **Breadth First Search (BFS) algorithm was successfully implemented in Prolog**, and the shortest path from the start node to the goal node was obtained.

28. Forward Chaining in Prolog

Aim

To write a Prolog program to implement **Forward Chaining** using facts and rules.

Algorithm

1. Define the initial facts.
2. Define rules to derive new facts.
3. Apply rules whose conditions are satisfied.
4. Continue deriving facts until the goal is reached.

Program

% Initial facts

fact(a).

fact(b).

% Rules for forward chaining

rule(c) :-

fact(a),

fact(b).

rule(d) :-

rule(c).

rule(e) :-

rule(d).

% Query to derive the facts

forward(X) :-

fact(X).

forward(X) :-

rule(X).

Required Queries

Query 1: Check the initial fact

?- forward(a).

Output:

true.

Query 2: Derive C from A and B

?- forward(c).

Output:

true.

Query 3: Derive D from C

?- forward(d).

Output:

true.

Query 4: Derive E from D

?- forward(e).

Output:

true.

Query 5: Check all derived facts

?- forward(X).

Sample Output:

X = a ;

X = b ;

X = c ;

X = d ;

X = e.

Result

Thus, the **Forward Chaining algorithm** was successfully implemented in Prolog, where new conclusions are derived from the given facts and rules.

29. Backward Chaining in Prolog

Aim

To write a Prolog program to implement **Backward Chaining** using facts and rules.

Algorithm

1. Start with the goal.
2. Check whether the goal is a known fact.
3. If not, find a rule that can produce the goal.
4. Check whether the conditions of the rule are true.
5. Continue backward until the facts are reached.
6. If all conditions are satisfied, the goal is proved.

Program

% Facts

parent(ram, suresh).

parent(suresh, raju).

% Rules

ancestor(X, Y) :-

parent(X, Y).

ancestor(X, Y) :-

parent(X, Z),

ancestor(Z, Y).

% Query rule

backward(X, Y) :-

ancestor(X, Y).

Required Queries

Query 1: Check direct ancestor

?- backward(ram, suresh).

Output:

true.

Query 2: Check indirect ancestor

?- backward(ram, raju).

Output:

true.

Query 3: Find the ancestor of Raju

?- backward(X, raju).

Sample Output:

X = ram ;

X = suresh.

Query 4: Check an invalid relationship

?- backward(raju, ram).

Output:

false.

Result

Thus, the **Backward Chaining algorithm** was successfully implemented in Prolog, where the system starts with a goal and works backward through rules and facts to prove it.

30. Create a Web Blog Using WordPress to Demonstrate Anchor Tag and Title Tag

Aim

To create a simple **web blog using WordPress** and demonstrate the use of an **Anchor Tag** and **Title Tag**.

Procedure / Algorithm

1. Open WordPress and log in to the dashboard.
2. Create a new blog post or page.
3. Add a suitable title to the blog.
4. Insert content about a topic.
5. Create an **anchor link** to navigate to another section or webpage.
6. Add a **title attribute** to provide additional information when the user hovers over the link.
7. Publish the blog.
8. Test the anchor link in the browser.

Example Blog Content

Blog Title

Welcome to My Technology Blog

Content

Welcome to my blog. This blog provides information about technology, programming, and web development.

Anchor Tag Example

```
<a href="https://www.google.com" title="Visit Google Website">
```

Click Here to Visit Google

```
</a>
```

Internal Anchor Link Example

```
<a href="#about">Go to About Section</a>
```

```
<h2 id="about">About</h2>
```

```
<p>This is the About section of my blog.</p>
```

Output

A WordPress blog page is created successfully with:

- A **blog title** displayed at the top.
- An **anchor link** that redirects the user to another webpage or section.
- A **title attribute** that displays additional information when the user hovers over the link.

Result

Thus, a **web blog was successfully created using WordPress**, demonstrating the use of the **Anchor Tag (<a>)** and the **Title Tag/Title attribute**.